



**How do we build reusable tools in Python that work like built-in functions such as `range()`?**

# Objectives

**In this session, we will explore:**

- <> How to **define functions** to create reusable code blocks.
- The principles of **modular programming** for better organization.

Functions and modular programming are especially useful in data science because they allow us to apply the same transformation or analysis to large datasets efficiently.

# Functions and Modular Code

## What Are Functions?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code over and over, we can define it once and use it multiple times.

Modular & Organized

Easier to Debug

### Python Implementation

```
def greet():  
    print("Hello, welcome to Python!")
```

# Define using the 'def' keyword

```
greet()
```

# Call function using its name  
# followed by parentheses

# Why Use Functions?

Functions provide several key advantages in software development:



## Reducing Repetition

Write code once, use it many times. This minimizes redundancy and code bloat.



## Improving Readability

Break down complex problems into smaller, manageable parts that are easy to understand.



## Making Debugging Easier

Errors are easier to find and fix when the code is well-organized into discrete logic units.



## Encouraging Reusability

Functions can be used across multiple programs, allowing for efficient modular design.

# Defining and Calling Functions

Functions can take parameters (inputs) and return values (outputs).

## Example 1: Function with Parameters

```
def greet_user(name):  
    print(f"Hello, {name}! Welcome to Python.")
```

Now, when we call the function and pass in a name, it personalizes the message:

```
greet_user("Alice")  
# Output: Hello, Alice! Welcome to Python.
```

# Returning Values from Functions

Some functions return values instead of just printing them.

## Example 2: Function That Returns a Value

```
def square(num):  
    return num * num
```

Calling the function stores the result in a variable:

```
result = square(4)  
print(result) # Output: 16
```

# Functions with Multiple Parameters

We can also create functions that take multiple inputs.

## Example 3: Adding Two Numbers

```
def add_numbers(a, b):  
    return a + b
```

Call the function with two numbers:

```
sum_result = add_numbers(5, 7)  
print(sum_result) # Output: 12
```

# Default Parameter Values

You can set a default value for a parameter so the function works even if no argument is provided.

## Example 4: Function with a Default Parameter

```
def greet(name="Student"):
    print(f"Hello, {name}!")
```

Calling the function with and without arguments:

```
greet() # Output: Hello, Student!
```

```
greet("Bob") # Output: Hello, Bob!
```



# Modular Code: Breaking Down Problems

Breaking code into smaller functions makes it modular and easier to manage.

## Example 5: Modular Code for a Calculator

Instead of writing one long function, we break it into smaller functions:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b == 0: return "Error! Division by zero."  
    return a / b
```

We can now reuse these functions to create a calculator.

# Summary: Key Takeaways

## Functions

Functions make code reusable and organized, acting as the building blocks of efficient programs. Functions also allow us to build scalable, reusable solutions to complex problems.



## Parameters

Parameters allow functions to accept input values, making them versatile and dynamic.



## Return Values

Return values allow functions to send results back to the main program or other functions.



## Modular Code

Breaking problems into smaller functions makes code easier to understand, manage, and debug.

**You just broke a complex program into smaller, reusable pieces.**

**This is the same approach used in real-world systems, where large problems are solved by combining smaller, well-designed functions.**

# Activity: Writing Functions



## Objective

In this activity, you will practice writing and using functions to create **reusable and modular code**.

Key focus areas:

- Defining functions with and without parameters
- Implementing return values
- Integrating functions into small programs

# Part 1: Defining and Calling a Simple Function

**Task:** Write a function that prints a greeting message.

1

**Step 1:** Open your Jupyter Notebook.

2

**Step 2:** Define a function named `greet()` that prints "Hello! Welcome to Python."

3

**Step 3:** Call the function to see the output.

# Example: Simple Python Function

## Python Code Implementation

```
def greet():  
    print("Hello! Welcome to Python.")  
  
# Call the function  
greet()
```



## Expected Output

```
> Hello! Welcome to Python.
```

## Part 2: Function with Parameters

**Task:** Modify the greeting function so that it takes a name parameter and prints a personalized greeting.

**1 Step 1:** Define a function `greet_user(name)` that takes a name parameter.

**2 Step 2:** Use `name` inside the function to customize the message.

**3 Step 3:** Call `greet_user()` with different names to test it.

# Example: Function with Parameters

## Python Code Implementation

```
def greet_user(name):  
    print(f"Hello, {name}! Welcome to Python.")  
  
# Call the function with different names  
greet_user("Alice")  
greet_user("Bob")
```



## Expected Output

```
> Hello, Alice! Welcome to Python.  
> Hello, Bob! Welcome to Python.
```

## Part 3: Function with Return Value

**Task:** Write a function that calculates the square of a number and returns the result.

- Step 1:** Define a function `square(num)` that takes one parameter.
- Step 2:** The function should return the square of num.
- Step 3:** Call the function and store the result in a variable.
- Step 4:** Print the result.



# Example: Function with Return Value

## Python Code Implementation

```
def square(num):  
    return num * num  
  
# Call the function and store the result  
result = square(5)  
print(result)
```



## Expected Output

```
> 25
```

# Part 4: Writing a Function with Multiple Parameters

**Task:** Write a function that adds two numbers and returns the result.

- Step 1:** Define a function `add_numbers(a, b)` that takes two parameters.
- Step 2:** Return the sum of a and b.
- Step 3:** Call the function with different values and print the results.

# Example: Function with Return Value

## Python Code Implementation

```
def add_numbers(a, b):  
    return a + b  
  
# Test the function  
sum1 = add_numbers(3, 7)  
sum2 = add_numbers(10, 20)
```



## Expected Output

```
> print(sum1) # Output: 10  
> print(sum2) # Output: 30
```

# Part 5: Writing a More Complex Function

**Task:** Write a function that calculates the area of a rectangle and returns the result.

- Step 1:** Define a function `rectangle_area(width, height)`.
- Step 2:** The function should return `width * height`.
- Step 3:** Call the function with different values.

# Example: Function with Return Value

## Python Code Implementation

```
def rectangle_area(width, height):  
    return width * height  
  
# Test the function  
area1 = rectangle_area(5, 10)  
area2 = rectangle_area(3, 7)
```



## Expected Output

```
> print(area1) # Output: 50  
> print(area2) # Output: 21
```

# Part 6: Challenge – Function with Conditional Logic

**Task:** Write a function that checks if a number is even or odd.

- Step 1:** Define a function `is_even(number)`.
- Step 2:** If number is even, return `"Even"`. Otherwise, return `"Odd"`.
- Step 3:** Call the function with different numbers and print the results.

# Example: Function with Conditional Logic

## Python Code Implementation

```
def is_even(number):  
    if number % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"  
  
# Test the function
```



## Expected Output

```
> print(is_even(4)) # Output: Even  
> print(is_even(7)) # Output: Odd
```

# Reflection & Submission

## Reflection Questions

- What are the benefits of using functions in your code?
- How do parameters and return values make functions more useful?
- How can functions help break down complex programs into smaller parts?

## Final Requirement



- Save and submit your completed notebook



# Activity: Automate the Boring Stuff

## Chapter 3: Functions

Take some time to see a rundown of how to write functions. This is a vital technique that can save you a lot of time and stress.

Instead of creating new code for every problem, you can use the same code over and over again by calling a function.



### Required Reading

[Click here to access Chapter 3 Link](#)